Prolog

A language for logic progamming

Prolog: LOGic PROgramming

- 1972 : created by Alain Colmerauer et P. Roussel à Marseille
  - For natural-language processing and automated theorem proving
- 1977: first compiler par D.H. Warren (Edinburgh Prolog)
  - with standardized syntax and semantics
- 1980: Borland Turbo Prolog
- 1995: ISO Prolog

Prolog and Classic AI

- Key concepts
  - Representation: Horn clauses
  - Fundamental operation: Unification
  - Execution model: Solution tree search
- Especially suited for:
  - Natural language processing
  - Classic expert systems
  - Symbolic reasoning
  - Knowledge representation

Prolog is descriptive and prescriptive

- Prolog is both a descriptive language (facts and relations) and a prescriptive one (inference)
  - It makes it possible to represent knowledge and to manipulate it
- Problems are resolved
  - For a domain (a set of objects)
  - And knowledge about the domain (the known relations between the objects)

A Prolog Program

- Specify facts
  - Unquestionably true statements about the domain of study
- Define rules
  - Rules allow new facts to be derived
- Ask questions
  - Questions that can be answered by the Prolog interpreter.
- *Prolog is based on first-order logic*

First-order logic

- Symbolic framework to describe a domain that includes:
  - Variables that range over individual objects
  - Predicates that express properties of objects or relations between them
  - Functions that map objects to other objects
  - Logical connectives such as $\wedge$, $\vee$, $\neg$
  - Quantifiers:
    - Universal: $\forall x$ ("for all x")
    - Existential: $\exists x$ ("there exists an x")
- "first-order" logic applies only to individual objects, not to predicates or sets.
  - Second-order logice authorizes quantifiers over functions or predicates

$$\forall P \forall x (Px \vee \neg Px)$$

Prolog Programming

- Programmers specify the What not the How
  - Highly expressive language
  - Rapid development (to some extent)
  - Requires solid mastery of predicate logic
  - Demands a good understanding of the language's internal mechanisms

Prolog data and variables

- Constants or atoms
  - represented by a character string beginning with a lowercase letter.
  - Numbers can be integers or floating-point values.
- Variables
  - written as a single letter or a name beginning with an uppercase letter.
  - can express a property concerning a category of objects
  - you also use variables to query Prolog

Prolog Relations

- A Prolog relation is a property that links a certain number of objects.
  - It can represent a connection between objects or a property of a single object

    * Example: *John is the father of Paul*

Prolog facts

- A **fact** is a basic assertion that something is true in the domain being modeled.
  - A fact names a relation and specifies the objects that participate in it.

  * *All human being are mortal*
  * *Socrates is mortal*

Predicates and formulas

- In Prolog relations are expressed using predicates
- Predicates have:
  - a name
  - A number of arguments
- A formula is a predicate applied to terms

  `father(john, paul)`
  - This is an atomic formula

Predicate arity

- The **arity** of a predicate is simply the number of arguments it takes
  - Unary predicate:
    - *human(X).*
  - Binary predicate:
    - *father(X,Y)*

human/1 **father/2**

Compound formula

- A compound formula is built using logical connectives
  - Fact (atomic):

```
human(socrates).
```

  - Rule (compound):
    - *If X is a human then X is mortal*
    - In Prolog « **if** » is written as « :- »

```
mortal(X) :- human(X).
```

Prolog Query

- In Prolog a query is a request to solve a **goal**
- A query triggers a proof search
- If the query contains variables, Prolog will try to assign values to these variables
  - Is Socrates mortal?
  - A question is expressed using the prompt « ?- »

```
?- mortal(socrates)
```

Horn clauses

- General form : F :- F1, F2,…, Fn.
  - which means « F if F1 and F2 and…and Fn »
  - F : must be an atomic formula
  - Fi : sequence of atomic formulas
- F is the head of the clause
- F1, F2,…, Fn is the body or tail of the clause
- To proof F, the goal is to demonstrate F1, F2,…, and Fn.
  - i.e. if F1, F2,…, Fn are true then F is automatically true
- A predicate is defined by facts and rules
- The Horn clauses are the only valid formulas in Prolog

Facts, rules and queries

- A fact is a Horn cluase without tail
- A query is a Horn clause without head
- A rule is a general Horn clause

- Prolog is essentially a programming language built on **Horn-clause logic**

The duck is a bird. Birds fly. Does the duck fly?

Closed world Hypothesis

- *Anything that cannot be proven true from the facts and rules is assumed to be false.*

Example

```
love(jean,sandra).
love(jean,paulette).
love(paulette,sam).
love(bruno,sandra).
drink(paulette,vin).
drink(bruno,jus).
drink(sam,biere).
smoke(bruno).
smoke(sandra).
smoke(sam).
```

…example

```
dance(jean,paulette):- drink(paulette,wine).
dance(bruno,sandra):- love(bruno,sandra), drink(sandra,water).
dance(sam,paulette):- love(paulette,sam),drink(sam,beer),
                                    \+smoke(paulette).
?-love(jean,paulette)
yes
?-drink(bruno,beer)
no
?-dance(bruno,sandra)
no
?-dance(sam,paulette)
yes
```

…example

```
?- listing(love)
love(jean,sandra).
love(jean,paulette).
love(paulette,sam).
love(bruno,sandra).
?- drink(Who,beer)
Who=sam
?-drink(jean,What)
no
```

```
?-aime(jean, Who)
Who =sandra;
Who =paulette;
No
?-dance(X,Y)
X=jean, Y=paulette;
X=sam, Y=paulette;
no
```

…example

```
sick(X):- smoke(X), drink(X,Y), alcool(Y).
alcool(beer).
alcool(wine).

?- sick(X)
X=sam
```

Another example

* *The goat is a herbivorous animal.*
* *The wolf is a carnivorous animal.*
* *Carnivorous animals are cruel.*
* *A herbivorous animal eats grass.*
* *A carnivorous animal eats herbivorous animals.*

- Question: Is there a cruel animal and what does it eat ?

And / Or

grandfather(X,Y):-father(X,Z), father(Z,Y).
grandfather(X,Y):- father(X,Z), mother(Z,Y).

The scope of a variable is limited to the formula:

grandfather(X,Y):- father(X,Z), father(Z,Y).
grandfather(X,Z):- father(X,Y), mother(Y,Z).

Prolog program

- Prolog program: a collection of predicates (Horn clauses)
- The order of the declarations is not relevant
- Predicates can be defined from other predicates

  * Example: defining child/2 from child/3
    * *child(C,Father,Mother)*
    * *child(C,Parent) :- child(C,Parent,_); child(C,_,Parent).*

Prolog demonstration

- It is the process of showing how a query is proven using the rules and facts of a Prolog program.
- A demonstration typically includes:
  - the initial query (goal clause)
  - the selected goal at each step
  - the clause used to resolve it
  - the substitution (unifier) applied
  - the new goal list after resolution

Combining (or chaining) clauses in demonstration

- Horn clauses can be combined to create new clauses:

  m :- a,b,c,d,e.

  d:- f,g,h.

  are combined to form:

  m,~~d~~ :- a,b,c,~~d~~,e,f,g,h.

Unification

- It is the process of making two terms identical by finding a consistent set of variable bindings
  - It is a built-in pattern-matching mechanism
  - Example :

```
brother(X,Y) :- man(X), child(X,Z), child(Y,Z), X\=Y.
?- brother(paul,Qui)
```

  - Successful unification:

    `{X=paul, Qui=Y}`

  - Unification may also fail

    `e(X,X)` and `e(2,3)` cannot be unified

Unification predicate

- Unification operator : « = »

```
* a(B,C) = a(2,3).
YES {B=2, C=3}

* a(X,Y,L) = a(Y,2,carole).
YES {X=2, Y=2, L=carole}

* a(X,X,Y) = a(Y,u,v).
  NO
```

Demonstration example

```
can_fly(X):-bird(X).
bird(duck).

?- bird(duck)
?- can_fly(duck)
```

- The query is the initial goal
- Prolog search for finds a clause whose head unifies with that goal

Demonstrating Negation: not ou \+

- If `F` is a clause then its negation is `not(F)`
- To demonstrate `not(F)`, Prolog will try to demonstrate `F`
  - If `F` is demonstrated then `not(F)` has failed
  - If `F` cannot be demonstrated, then `not(F)` is considered to have been demonstrated

Some examples

```
?- not(X=1).
no
?- X=0, not(X=1).
X= 0
?- not(X=1), X=0.
no
```

Negation and variables

- Demonstrating a negation should never assign a value to a variable used after

```
single_student(X):-
      not(married(X)), student(X).

student(joe).
married(jean).

?- single_student(joe).
yes
?- single_student(jean).
no
?- single_student(X).
no
```

Prolog SLD – resolution

- Selective
  - Prolog selects one goal at a time to resolve.
  - it always chooses the leftmost goal in the list.
- Linear
  - Resolution proceeds linearly, each step is derived from the previous one.
- Definite
  - Prolog programs consist of **definite Horn clauses.**

Demonstration steps

- A demonstration step is a single operation in the SLD-resolution process where:
  - Prolog selects the leftmost goal in the current goal list.
  - It finds a clause whose head unifies with that goal.
  - It applies unification if required.
  - It replaces the selected goal with the body of the clause (possibly empty).
  - The result is a new goal list, which becomes the next step.

A family example

```
man(john).
man(peter).
man(marc).
woman(mary).
woman(louise).

parent(john,peter).
parent(peter,marc).
parent(peter,louise).
parent(mary,marc).
parent(mary,louise).
```

```
?- parent(peter,C).
C = marc ;
C = louise.

?- parent(P,louise).
P = peter ;
P = mary.

?- parent(john,peter).
true.

?- parent(john,_).
true.

?- parent(marc,_).
false.
```

```
?- X = peter.
X = peter.

?- peter = X.
X = peter.

?- parent(marc,sam)=parent(marc,X).
X = sam.
```